Profiling Analysis of the Hybrid CPU+GPU Mandelbrot Renderer

Matias Saavedra

Friday 10th July, 2026

Binary: `0f782fd` (tag `binary-v0-buggy`) — original code, with the min/max tracking bug in `MandelRegion::examine()`.

Resumen

This report documents the instrumentation added to `mandelHybrid`, a hybrid CPU+GPU Mandelbrot set renderer that uses adaptive region splitting as its load-balancing strategy. Two complementary profiling mechanisms were introduced: NVTX (NVIDIA Tools Extension) annotations, which label regions of CPU code so that Nsight Systems can render them as colored spans on a per-thread timeline, and CUDA event timing, which uses GPU-side hardware timestamps to measure kernel execution and device-to-host transfer time independently of any CPU clock. The results show that the GPU thread is approximately 40× faster per region than a single CPU thread, naturally pulling 77% of all work-queue items. All 12 threads (1 GPU + 11 CPU) finish within roughly 2 seconds of each other, demonstrating that the dynamic work queue achieves good load balance. The dominant finding is that `cudaMemcpy` spends 99.9% of its reported time waiting for the GPU kernel to finish—the actual device-to-host data transfer takes only 15 µs on average, not 13.4 ms as the CUDA API summary suggests.

1. Program Overview

`mandelHybrid` renders a sequence of Mandelbrot frames using a shared work queue. One `CalcThr` thread (thread 0) drives the GPU; all remaining threads compute on the CPU. Every thread calls `MandelRegion::examine()` in a loop, pulling items from the queue until it is empty. `examine()` evaluates the four corners of a region: if the corners are uniform (the spread of iteration counts is small relative to the `diffThresh` parameter) or the region is smaller than `pixelSizeThresh` pixels, it calls `MandelRegion::compute()` immediately; otherwise it splits the region into four equal quadrants and pushes them all back onto the queue.

The profiling run rendered 100 frames at 1920×1080 pixels on a NVIDIA GeForce GTX 1660 Ti Max-Q using 12 threads (1 GPU thread + 11 CPU threads), which is the machine's physical core count.

2. Instrumentation Changes

2.1. Build System (Makefile)

The only build-system change was adding `-ldl` to the linker flags:

Código 1: Makefile — new linker flag.

```
# NVTX3 (CUDA 11+) is header-only: it dlopen()s the profiler's runtime
# library when Nsight Systems is active, so no -lnvToolsExt is needed.
# -ldl provides the dlopen symbol that the NVTX3 headers use internally.
CC_LINK_FLAGS = -lm -lstdc++ $(QT_LIBS) -lpthread -ldl
```

Why -ldl? The NVTX3 API (CUDA 11+) is entirely header-only. When the program starts under Nsight Systems, the profiler injects a shared library (`libNvtxInjection64.so`) into the process using `LD_PRELOAD`. The NVTX3 headers then call `dlopen()` at the first `nvtxRangePush` to locate the injected library and register the annotation. Without linking `libdl`, the linker cannot resolve `dlopen` and the build fails. When the program runs without Nsight Systems, the `dlopen` call simply finds nothing and all NVTX calls become no-ops with negligible overhead.

In CUDA 10 and earlier there was a real shared library `libnvToolsExt.so` to link against. CUDA 11 replaced it with the header-only NVTX3 design—the `nvtx3/nvToolsExt.h` header ships inside the CUDA toolkit at `$CUDAINST/include/nvtx3/`.

2.2. NVTX Annotations

2.2.1. What NVTX Is

NVTX (NVIDIA Tools Extension Library) is a C API for annotating host-side code with ranges and markers. A range is an interval opened with `nvtxRangePush("label")` and closed with `nvtxRangePop()`. Ranges stack: if you push inside an existing range, the inner range appears nested under the outer one. Nsight Systems records the wall-clock timestamps of every push and pop, associates them with the calling thread, and renders them as colored bars on a per-thread lane in the timeline view. This lets you see—at a glance and without changing the program's logic—which thread is doing what, for how long, and how CPU and GPU work interleave.

2.2.2. Adding the Header

The same header is included in both the CUDA translation unit and the plain C++ translation unit:

Código 2: kernel.cu — NVTX header.

```
// nvToolsExt.h lives under nvtx3/ in CUDA 11+ installations.
#include <nvtx3/nvToolsExt.h>
```

Código 3: mandelregion.cpp — NVTX header.

```
// Including it in a plain C++ translation unit is fine;
// nvcc is not required to use the NVTX3 CPU-side API.
#include <nvtx3/nvToolsExt.h>
```

2.2.3. Annotating examine()

examine() is the innermost work-queue loop body executed by every thread for every task. A range was added around the entire function:

Código 4: mandelregion.cpp — examine() annotation.

```
void MandelRegion::examine(WorkQueue &q, bool onGPU)
{
   // Wraps the full decision cycle: corner evaluation + compute or split.
   // In Nsight Systems this appears on the calling thread's row, letting
   // you count how many examine() calls each thread handles.
   nvtxRangePush("examine region");

   // ... corner evaluation and compute/split logic (unchanged) ...

   nvtxRangePop(); // closes "examine region"
}
```

Because examine() calls compute(), the CPU and GPU compute ranges (described below) appear nested inside "examine region" in the timeline. The gap between the outer and inner range represents pure overhead: corner-iteration calls and queue operations.

2.2.4. Annotating compute() — CPU Path

Código 5: mandelregion.cpp — CPU compute annotation.

```
void MandelRegion::compute(bool onGPU)
{
   // ...
   else // CPU path
   {
      // Nsight Systems shows this as a colored bar on the calling
      // thread's lane alongside CUDA API activity on thread 0.
      nvtxRangePush("CPU region compute");

      // ... existing pixel loop ...

      nvtxRangePop(); // closes "CPU region compute"
   }
}
```

2.2.5. Annotating hostFE() — GPU Path

Código 6: kernel.cu — GPU compute annotation.

```
extern "C" void hostFE(double upperX, double upperY, double lowerX,
                       double lowerY, int resX, int resY,
                       unsigned int **pixels, int *currpitch, int MAXITER)
{
   // Visible in Nsight Systems CPU timeline alongside the CUDA API calls
   // automatically recorded by the driver (kernel launch, memcpy).
```

```
    nvtxRangePush("GPU region compute");

    // ... kernel launch and memcpy (see Section 2.3) ...

    nvtxRangePop(); // closes "GPU region compute"
}
```

2.3. CUDA Event Timing

2.3.1. What cudaEvent_t Is

A `cudaEvent_t` is an opaque handle to a hardware timestamp on the GPU. The GPU has its own free-running clock, independent of any CPU clock. When you call `cudaEventRecord(event)`, the CUDA runtime enqueues a "stamp yourself" command into a CUDA stream. The GPU executes this command in stream order: it will not record the timestamp until all previously enqueued work in that stream has completed. The CPU returns from `cudaEventRecord` immediately—the stamping happens asynchronously.

After the work you want to measure is done, you call `cudaEventSynchronize(event)`, which blocks the CPU until the GPU has written that specific timestamp. Then `cudaEventElapsedTime(&ms, start, stop)` reads the delta in milliseconds from the GPU hardware. This approach is more accurate than a CPU-side timer because:

- The GPU runs asynchronously; a CPU timer started before a kernel launch would include driver dispatch latency, not just kernel execution time.
- `cudaMemcpy` with `cudaMemcpyDeviceToHost` is a blocking CPU call that first waits for any pending GPU work to finish and then performs the transfer. A CPU timer around it measures kernel wait time + transfer time, not transfer time alone. CUDA events placed around just the transfer report only the transfer.

2.3.2. Declaring Four Events and Accumulators

Four events are declared as file-scope statics in `kernel.cu` so they persist across all calls to `hostFE()`:

Código 7: kernel.cu — event handles and accumulators.

```
static cudaEvent_t evKernelStart, evKernelStop;
static cudaEvent_t evMemcpyStart,  evMemcpyStop;

// Cumulative totals across all hostFE() calls on thread 0.
static double totalKernelMs = 0.0;
static double totalMemcpyMs = 0.0;
static long   totalRegions  = 0;
```

Using four events—two around the kernel, two around the memcpy—lets us separate kernel execution time from transfer time. Without them we could only measure the combined time reported by the blocking `cudaMemcpy` call.

2.3.3. Creating Events in CUDAmemSetup()

Código 8: kernel.cu — CUDAmemSetup(), event creation.

```
extern "C" unsigned int *CUDAmemSetup(int maxResX, int maxResY)
{
   // ... existing cudaMallocPitch and cudaHostAlloc ...

   // Create CUDA events once; reuse them for every kernel call.
   // cudaEventCreate allocates GPU-side timer resources.
   CUDA_CHECK_RETURN(cudaEventCreate(&evKernelStart));
   CUDA_CHECK_RETURN(cudaEventCreate(&evKernelStop));
   CUDA_CHECK_RETURN(cudaEventCreate(&evMemcpyStart));
   CUDA_CHECK_RETURN(cudaEventCreate(&evMemcpyStop));

   return h_res;
}
```

CUDAmemSetup() is called once before any rendering begins, making it the right place to allocate GPU-side timer resources. Creating them here avoids paying the allocation cost (a GPU round-trip) on every one of the 3,543 kernel launches.

2.3.4. Recording Timestamps in hostFE()

Código 9: kernel.cu — hostFE(), full event timing block.

```
// Record a GPU timestamp immediately before the kernel launch.
// The stamp executes after all previously-queued work finishes.
CUDA_CHECK_RETURN(cudaEventRecord(evKernelStart));

mandelKernel<<<grid, block>>>(d_res, upperX, upperY,
                        stepX, stepY, resX, resY, ptc, MAXITER);

// Stamp the moment the kernel finishes (still async -- resolved below).
CUDA_CHECK_RETURN(cudaEventRecord(evKernelStop));

// Stamp before the device-to-host copy.
CUDA_CHECK_RETURN(cudaEventRecord(evMemcpyStart));

CUDA_CHECK_RETURN(
   cudaMemcpy(h_res, d_res, resY * ptc, cudaMemcpyDeviceToHost));

CUDA_CHECK_RETURN(cudaEventRecord(evMemcpyStop));

// Block the CPU until the GPU has written all four timestamps.
// After this call, cudaEventElapsedTime gives accurate durations.
CUDA_CHECK_RETURN(cudaEventSynchronize(evMemcpyStop));

float kernelMs = 0.0f, memcpyMs = 0.0f;
CUDA_CHECK_RETURN(
   cudaEventElapsedTime(&kernelMs, evKernelStart, evKernelStop));
CUDA_CHECK_RETURN(
   cudaEventElapsedTime(&memcpyMs, evMemcpyStart, evMemcpyStop));
```

```
28
29 totalKernelMs += kernelMs;
30 totalMemcpyMs += memcpyMs;
31 totalRegions++;
32
33 // Per-region line printed to stderr.
34 fprintf(stderr,
35    "[GPU region %4ld] size %4dx%4d  kernel %.3f ms  D->H %.3f ms\n",
36    totalRegions, resX, resY, kernelMs, memcpyMs);
```

The key subtlety is that `cudaEventRecord(evKernelStop)` is called on the CPU immediately after the kernel launch returns, not after the kernel finishes. Because the default CUDA stream serializes all operations, the GPU will record `evKernelStop` only after `mandelKernel` completes. Similarly, `evMemcpyStart` is stamped after the kernel finishes but before the bytes start moving, `evMemcpyStop` after the last byte arrives. The `cudaEventSynchronize(evMemcpyStop)` call is the single point where the CPU blocks, ensuring all four timestamps have been committed before `cudaEventElapsedTime` reads them.

2.3.5. Printing the Summary in CUDAmemCleanup()

Código 10: kernel.cu — CUDAmemCleanup(), summary and teardown.

```
1  extern "C" void CUDAmemCleanup()
2  {
3     // Accumulated GPU-side timing. These numbers exclude host-side overhead.
4     fprintf(stderr,
5        "[GPU profiling summary]\n"
6        "  Regions computed on GPU : %ld\n"
7        "  Total kernel time       : %.3f ms  (avg %.3f ms/region)\n"
8        "  Total D->H memcpy time  : %.3f ms  (avg %.3f ms/region)\n",
9        totalRegions,
10       totalKernelMs, totalRegions ? totalKernelMs / totalRegions : 0.0,
11       totalMemcpyMs, totalRegions ? totalMemcpyMs / totalRegions : 0.0);
12
13    cudaEventDestroy(evKernelStart);
14    cudaEventDestroy(evKernelStop);
15    cudaEventDestroy(evMemcpyStart);
16    cudaEventDestroy(evMemcpyStop);
17
18    CUDA_CHECK_RETURN(cudaFreeHost(h_res));
19    CUDA_CHECK_RETURN(cudaFree(d_res));
20 }
```

2.4. CPU Thread Timing with clock_gettime

2.4.1. Thread-Local Accumulators

Código 11: mandelregion.cpp — thread-local state.

```cpp
#include <time.h>

// Each thread gets its own independent copy, initialized to zero.
// No mutex is needed because no two threads share the same copy.
static thread_local long   cpuRegionCount = 0;
static thread_local double cpuTotalMs     = 0.0;

static inline double elapsedMs(const struct timespec &t0,
                               const struct timespec &t1)
{
   return (t1.tv_sec  - t0.tv_sec)  * 1e3
        + (t1.tv_nsec - t0.tv_nsec) * 1e-6;
}
```

The thread_local storage class ensures each CalcThr thread has a private (cpuRegionCount, cpuTotalMs) pair. With 11 CPU threads running concurrently, a shared counter would require a mutex; thread_local eliminates all synchronization from the hot path.

CLOCK_MONOTONIC is used instead of CLOCK_REALTIME because the monotonic clock never jumps backwards due to NTP corrections or daylight saving changes.

2.4.2. Bracketing the Pixel Loop in compute()

Código 12: mandelregion.cpp — CPU timing in compute().

```cpp
else // CPU path
{
   nvtxRangePush("CPU region compute");

   struct timespec t0, t1;
   clock_gettime(CLOCK_MONOTONIC, &t0);

   for (int i = 0; i < pixelsX; i++)
      for (int j = 0; j < pixelsY; j++)
      {
         // ... diverge() and setPixel() (unchanged) ...
      }

   clock_gettime(CLOCK_MONOTONIC, &t1);
   double ms = elapsedMs(t0, t1);
   cpuTotalMs += ms;
   cpuRegionCount++;

   fprintf(stderr,
      "[CPU region %4ld] size %4dx%4d  compute %.3f ms\n",
      cpuRegionCount, pixelsX, pixelsY, ms);

   nvtxRangePop();
}
```

2.4.3. Per-Thread Summary Function

A new static method was declared in **mandelregion.h** and implemented in **mandelregion.cpp**:

Código 13: mandelregion.cpp — printCPUSummary().

```
void MandelRegion::printCPUSummary()
{
   // GPU thread: cpuRegionCount == 0, nothing to print.
   if (cpuRegionCount == 0) return;
   fprintf(stderr,
      "[CPU profiling summary - this thread]\n"
      "  Regions computed on CPU : %ld\n"
      "  Total compute time      : %.3f ms  (avg %.3f ms/region)\n",
      cpuRegionCount,
      cpuTotalMs, cpuTotalMs / cpuRegionCount);
}
```

This function is called from **CalcThr::run()** once the work queue drains, meaning it executes exactly once per thread at the natural end of that thread's work:

Código 14: main.cpp — calling printCPUSummary at thread exit.

```
void CalcThr::run()
{
   MandelRegion *t;
   while ((t = que->extract()) != NULL)
   {
      t->examine(*que, isGPU);
      delete t;
   }
   // Work queue empty: print this thread's accumulated CPU timing.
   MandelRegion::printCPUSummary();
}
```

3. Results

The program was profiled with Nsight Systems using:

Código 15: Profiling command.

```
nsys profile --trace=cuda,nvtx,osrt -o report ./mandelHybrid spec.in
```

The **spec.in** file specifies 100 frames at 1920×1080 pixels. No thread count was passed, so the program defaulted to the machine's logical core count (12 threads: 1 GPU + 11 CPU).

3.1. NVTX Range Summary

Table 1 shows the aggregated NVTX range statistics reported by **nsys stats**. All times are summed across all threads and all instances.

Tabla 1: NVTX range summary (summed across all threads).

Range	Time%	Total (s)	Instances	Avg (ms)	Med (ms)	Max (ms)
`examine region`	51.0%	639.1	6,104	104.71	3.04	15,319
`CPU region compute`	45.2%	567.2	1,060	535.11	146.68	15,319
`GPU region compute`	3.8%	47.8	3,543	13.51	5.55	465.7

3.2. Per-Thread CPU Work Distribution

Table 2 shows the work each CPU thread received from the shared queue, sorted by total time descending. The data was extracted directly from the Nsight Systems SQLite database.

Tabla 2: CPU region distribution across the 11 CPU threads.

Thread	Regions	Total (s)	Avg (ms/region)
1	83	52.59	633.6
2	105	52.17	496.9
3	101	51.79	512.7
4	126	51.75	410.7
5	72	51.59	716.5
6	97	51.54	531.3
7	90	51.48	572.0
8	124	51.43	414.8
9	112	51.24	457.5
10	83	51.15	616.3
11	67	50.50	753.7
GPU thread	3,543	47.11	13.3

3.3. CUDA API and GPU Kernel Summary

Table 3 summarises the CUDA API calls from the driver's perspective (host-side), while Table 4 and Table 5 give the actual GPU-side times as reported by the CUPTI hardware counters.

Tabla 3: CUDA API summary (host-side, driver perspective).

API call	Time%	Calls	Avg	Total
`cudaMemcpy`	99.3%	3,543	13.4 ms	47.4 s
`cudaLaunchKernel`	0.2%	3,543	21.1 µs	74.6 ms
`cudaEventRecord`	0.2%	14,172	6.4 µs	90.3 ms
`cudaEventSynchronize`	0.0%	3,543	4.9 µs	17.4 ms

Tabla 4: GPU kernel execution time (CUPTI hardware counters).

Kernel	Instances	Total (s)	Avg (ms)	Med (ms)	Min (µs)	Max (ms)
`mandelKernel`	3,543	47.11	13.30	5.40	8.2	465.3

Tabla 5: GPU device-to-host memory transfer (CUPTI hardware counters).

Operation	Count	Total (s)	Total (MB)	Avg (µs)	Min (µs)	Max (µs)
D→H memcpy	3,543	0.053	599.9	15.0	10.7	160.6

4. Analysis

4.1. GPU is 40× Faster Per Region

Comparing the average per-region compute times:

- GPU: 13.5 ms average (from NVTX `"GPU region compute"`)
- CPU: 535 ms average (from NVTX `"CPU region compute"`, summed across all 11 CPU threads)

The GPU is roughly 40× faster per region. Because all threads compete for the same work queue, the GPU thread cycles back to pull a new region 40× more often, naturally absorbing 77% of all leaf regions (3,543 out of 4,603).

This ratio is not a tunable parameter—it is determined by the compute capability of the GPU versus the collective throughput of the CPU cores for the Mandelbrot iteration kernel.

4.2. Load Balance Is Good, But Not Perfect

The per-thread total compute times (Table 2) span 50.50 s to 52.59 s across the 11 CPU threads—a spread of only 4%. The GPU thread finishes its work in 47.1 s, roughly 3–5 s earlier than the last CPU thread. The dynamic work queue achieves its goal: no thread sits idle for long stretches while others have work.

The remaining 4% imbalance is not caused by a structural flaw in the scheduler. It arises entirely from per-region variance: some Mandelbrot regions near the set's boundary require far more iterations than the corner heuristic predicted. A thread that happens to pull such a region will take much longer on it than a thread pulling a uniform interior region, even if both regions have the same pixel count.

4.3. The D→H Memcpy Is Not the Bottleneck

This is the most surprising finding. The CUDA API summary reports `cudaMemcpy` as consuming 99.3% of all CUDA API time (13.4 ms average per call). However, the CUPTI hardware counters tell a different story: the actual device-to-host transfer takes only 15 µs on average and 53 ms total across all 3,543 copies. The discrepancy is explained by the semantics of `cudaMemcpy`:

1. `cudaMemcpy` with `cudaMemcpyDeviceToHost` is a blocking, synchronous call.
2. It first waits for all previously-enqueued GPU work to finish (i.e., it implicitly synchronizes the default stream).
3. Only then does it initiate the DMA transfer.

So the 13.4 ms reported by the CUDA API is almost entirely kernel execution time (confirmed by the GPU kernel summary: 13.3 ms average). The actual PCIe transfer consumes only 15 µs. At 599.9 MB across 3,543 copies the effective bandwidth is $599.9/0.053 \approx 11.3$ GB/s, which is healthy use of a PCIe 3.0 ×16 link (theoretical maximum: 16 GB/s). Optimizing memory layouts or using pinned memory more aggressively would have essentially no impact.

This finding also highlights a missed optimization: during those 13.4 ms where the CPU is blocked in `cudaMemcpy` (which is really waiting for the kernel), thread 0 (the GPU thread) cannot pull new work from the queue. Using asynchronous CUDA streams—launching the kernel into a non-default stream, overlapping the previous copy with the next kernel launch—would allow thread 0 to stay in the queue-pulling loop and increase GPU utilization.

4.4. High Per-Region Variance

- GPU regions: min 8.2 µs, avg 13.3 ms, max 465.3 ms (57,000× range)
- CPU regions: min 0.9 ms, avg 535 ms, max 15.3 s (17,000× range)

The 15.3 s CPU outlier is a single large, complex region where the four corners happened to agree (low iteration-count spread), so the heuristic classified it as uniform and did not split it. However, the interior contained many boundary pixels that required near-`MAXITER` iterations. This single region is responsible for almost the entire 4% thread imbalance. Lowering `diffThreshold` (currently 0.5) would force more splits on ambiguous regions, reducing worst-case outliers at the cost of a higher total number of work-queue items.

4.5. Examine Overhead Is Negligible

The total time attributed to `"examine region"` minus the time attributed to the two compute ranges gives the pure overhead of the adaptive subdivision logic (corner evaluation and queue operations):

$$639.1 - 567.2 - 47.8 \approx 24.1 \text{ s across } 6{,}104 \text{ calls} \approx 4 \text{ ms per call}$$

Four milliseconds per `examine()` call for up to four `diverge()` corner evaluations is a small overhead relative to the compute times involved.

4.6. Summary of Findings and Recommended Improvements

Tabla 6: Key findings and their implications.

Finding	Implication
GPU 40× faster per region	It naturally pulls 77% of work-queue items
All threads finish within ∼2 s of each other	Dynamic work queue achieves good load balance
D→H transfer: 15 µs avg, not 13.4 ms	PCIe bandwidth is not the bottleneck; kernel time dominates
GPU thread blocks 13.4 ms per region in `cudaMemcpy`	Async streams could recover this idle time
15.3 s CPU outlier region	Corner heuristic misclassified a complex region; lower `diffThreshold`
`examine()` overhead: ∼4 ms/call	Subdivision logic is cheap; not worth optimizing

Three concrete improvements follow from the analysis:

1. **Asynchronous CUDA streams.** Launch `mandelKernel` into a named stream and use `cudaStreamSynchronize` only when reading results. This lets thread 0 call `que->extract()` and begin preparing the next region while the previous kernel executes and copies, recovering the ∼13 ms per-region idle time.

2. **Tighter `diffThreshold`.** The default value of 0.5 is permissive. Reducing it to 0.2–0.3 would force more splits on ambiguous boundary regions, capping the worst-case per-region time and improving tail-latency balance across CPU threads.

3. **Largest-first work queue.** The `MandelRegion::Compare` functor already exists in `mandelregion.h` for this purpose. Replacing `WorkQueue`'s `deque` with a `priority_queue` (largest pixel area first) would give the GPU thread preferentially larger regions, which it handles fastest, and leave smaller regions for CPU threads, reducing the chance of a large outlier blocking a CPU thread near the end.